

# Sentence-BERT を用いた用途・予算別 PC 構成推薦システム

学籍番号 24026111, 氏名佐藤信晃

2026 年 8 月 1 日

## 1 課題設定

### 1.1 背景と解決する問題

PC を購入する際には、CPU、GPU、メモリ、ストレージ、電源容量など、多数の部品を用途に応じて選ぶ必要がある。しかし、初心者が「WQHD でゲームを快適に遊びたい」などの要求を持っていても、個々の部品名から適切な構成を判断することは難しい。また、同じ用途でも「動画制作」と「YouTube 用の映像編集」のように表現が異なるため、単純な文字列一致だけでは要求に近い構成を検索できない場合がある。

そこで本課題では、利用者が自然文で入力した用途と予算から、意味の近い PC 構成を推薦するプロトタイプを実装した。自然文の意味的な近さを扱うため、Sentence-BERT (SBERT) による文章埋め込みを中核技術として使用する。

一般的な PC 販売サイトでは、価格帯や部品名による絞り込みはできるが、利用者が持つ要求を部品条件へ言い換える作業は利用者自身に委ねられることが多い。例えば「配信しながらゲームをしたい」という要求から、CPU の並列処理性能、GPU 性能、メモリ容量の優先順位を初心者が決めることは容易ではない。本システムは、この要求と部品条件の間を、あらかじめ用途説明と結び付けた PC 構成を意味検索することで補う。最終的な購入を自動決定するものではなく、候補を数件に絞り、比較を始めるための意思決定支援を目的とする。

### 1.2 対象ユーザと要求

対象ユーザは、PC 部品について詳しくない学生、ゲーマー、動画制作者などである。主な要求を以下に示す。

- 専門用語を使わず、自然文で用途を入力できる。
- 指定した予算を超えない構成だけを比較できる。
- 推薦された CPU、GPU、RAM などを一覧で確認できる。
- 推薦理由と入力文に対する類似度を確認できる。

ここで重要なのは、予算を文章の類似度に混ぜず、独立した必須条件として扱う点である。用途が非常に近くても予算を超える候補を表示すると、推薦として実用性が低い。そのため、本システムでは最初に予算で候補集合を限定し、その集合内でのみ意味的な順位を決定する。一方、利用者の好みを完全に把握するには、ディスプレイ解像度、重視するソフトウェア、静音性などの追加情報も必要になる。本プロトタイプでは入力を簡潔に保つため、それらを一つの自然文に含める設計とした。

### 1.3 想定ユースケース

利用者は Web 画面に「Apex を高フレームレートで遊びたい」と入力し、予算を 22 万円に設定する。システムは予算内の候補だけを抽出し、入力文と各候補の登録用途との意味的類似度を計算する。そして、類似度の高い順に PC 構成、概算価格、推薦理由、予算残額を表示する。例えば、この入力では競技 FPS 向けの構成が上位になることを期待する。

別の利用場面として、大学生が「レポート作成が中心だが軽いゲームもしたい」と入力する場合がある。このとき高価なゲーム専用機を提示するのではなく、予算内で学生向けカテゴリが上位になることが望ましい。また、動画制作者が「YouTube 用の編集とゲームを一台で行いたい」と入力した場合には、単一用途ではなく複合用途を記述した候補を検索する必要がある。このように、製品名を直接指定しない複数の表現を同じ検索方式で扱えることを主要な利用価値とした。

## 2 技術概要

### 2.1 Sentence-BERT

BERT を用いて文の類似度を求める単純な方法では、文の組を一つずつモデルへ入力する必要があり、多数の候補との比較には計算量が大きい。Sentence-BERT は Siamese Network 構造を用いて、各文を固定長ベクトルへ変換し、ベクトル間の距離によって文の意味的類似度を高速に計算できるようにした手法である [1]。

本システムでは、日本語を含む 50 以上の言語を扱い、文や短い段落を 384 次元のベクトルへ写像する paraphrase-multilingual-MiniLM-L12-v2 を使用した [2]。PC 用途の登録文と利用者の入力文は、それぞれ同じ埋め込み空間のベクトルへ変換される。

### 2.2 モデルの選定理由

本システムでは、日本語の自由記述を扱うこと、CPU 環境でも実行可能な規模であること、追加学習を行わずにプロトタイプを構築できることをモデル選定の条件とした。英語専用のモデルでは、日本語の用途文に対して適切な埋め込みが得られる保証がない。一方、大規模な多言語モデルは高い表現能力が期待できるが、授業用 PC での読み込み時間やメモリ使用量が増え、対話的な検索画面には過剰となる可能性がある。そこで、多言語対応と軽量性のバランスを考え、12 層の MiniLM を基礎とする paraphrase-multilingual-MiniLM-L12-v2 を採用した。

このモデルは、入力文を 384 個の実数からなる一つのベクトルへ変換する。候補が 26 件の場合、コーパス埋め込みは  $26 \times 384$  個の実数で表現できるため、全候補をメモリ上に保持してもプロトタイプの規模では問題にならない。候補文は起動後に一度だけ変換し、検索時には入力文一件だけを変換する設計とした。これにより、候補数に比例してモデル推論を繰り返す必要がなく、候補追加後も同じ検索処理を利用できる。

モデルは PC 分野に特化して追加学習されたものではないため、GPU 型番やゲーム固有名詞の関係を常に正確に表現できるとは限らない。しかし本課題の目的は、既製の文章埋め込みモデルを実用システムのコアへ組み込み、その挙動を評価することである。したがって、追加学習によって評価データへ適合させず、事前学習済みモデルを固定して使用した。この判断により、評価結果は小規模コーパスと汎用埋め込みの組合せによる性能として解釈できる。

### 2.3 コサイン類似度

入力文の埋め込みを  $q$ 、候補文の埋め込みを  $d$  とすると、コサイン類似度は式 (1) で定義される。

$$\text{sim}(q, d) = \frac{q \cdot d}{\|q\|_2 \|d\|_2} \quad (1)$$

値が大きい候補ほど、入力文と意味的に近いと判断する。実装では埋め込み生成時に L2 正規化を行った後、`sentence_transformers.util.cos_sim`を用いて類似度を計算した。Sentence Transformers の公式文書でも、文埋め込み同士の意味的類似度の算出方法としてコサイン類似度が示されている [3]。

L2 正規化後のベクトルでは各ベクトルのノルムが 1 となるため、コサイン類似度は内積と等価になる。本実装ではライブラリ関数を使用しているが、順位付けの考え方は、入力ベクトルが指す方向と候補ベクトルが指す方向の近さを比較するものである。ベクトルの大きさではなく方向を見るため、文の長さが多少異なる候補も比較できる。ただし、類似度は性能の達成度や購入後の満足度を直接表す確率ではない。例えば 0.8 という値を「80% 満足する」と解釈することはできず、同じ候補集合内の相対的な順位付けに用いる指標である。

## 2.4 キーワード検索との違い

キーワード検索では、入力文と登録文が共有する語を数えることで候補を探せる。実装が単純で結果も説明しやすい反面、「映像を作りたい」と「動画編集」のように共通語が少ない言い換えを見落とす可能性がある。SBERT は文全体をベクトル化するため、表面的な文字列が一致しなくても、学習済みモデルが捉えた意味の近さを利用できる。PC 用途では、ゲーム名、解像度、制作作業など多様な表現が入力されるため、この性質が適していると考えた。

一方、数値や固有名詞の厳密な一致は意味検索だけでは保証されない。「16GB 以上」や「30 万円以内」のような条件は、文章に埋め込んで比較するより、構造化した列に対する数値判定の方が確実である。本設計が予算をフィルタ、用途を SBERT という二種類の処理に分けたのは、柔軟な条件と厳密な条件をそれぞれ適切な方法で扱うためである。

## 2.5 推薦用コーパス

推薦用コーパスは CSV 形式で独自に作成した 26 件の PC 構成である。各行は用途文、概算価格、CPU、GPU、RAM、SSD、電源、カテゴリ、推薦理由を持つ。ゲーム、配信、動画編集、3D 制作、画像生成 AI、機械学習、学生向けなど、異なる目的を含めた。

各用途文は、単に部品名を並べるのではなく、「誰が何をどの程度行いたいのか」が分かる一文として記述した。カテゴリは評価時の正解ラベルとして用い、推薦理由は検索後に人が候補を確認するための説明として用いる。埋め込みの計算対象は `use_case` 列だけであり、カテゴリ名や推薦理由を入力ベクトルとの比較へ含めていない。評価ラベルを検索文へ混入させず、用途説明の意味だけから順位を決めるためである。

26 件という規模は実サービスには小さいが、プロトタイプでは候補文を目視確認でき、誤推薦の理由を一件ずつ分析できる利点がある。また、同じ「ゲーム」でも競技 FPS、WQHD、4K、VR、配信併用などを分け、近接しやすい候補を意図的に含めた。これにより、全く異なる用途を区別するだけでなく、類似した用途間の順位も観察できるようにした。

本プロトタイプの目的は市場価格を厳密に見積もることではなく、文章埋め込みによる意味検索の動作を検証することである。したがって、価格と部品構成は固定した検証用データであり、実店舗の在庫や価格を外部 API から取得していない。

コーパスに含まれる 26 件の概算価格帯を図 1 に示す。候補は 10 万円台から 60 万円台まで存在し、特に 20 万円台と 30 万円台を中心に構成されている。これは、中価格帯のゲーム、配信、制作用途を複数比較できるようにしたためである。一方、50 万円以上の候補は少なく、高価格帯では用途の違いを細かく比較しにくいというデータ上の偏りがある。

# 3 システムの設計

## 3.1 入出力

入力は用途を表す自然文、予算、表示候補数の三つである。予算は 14 万円から 60 万円、候補数は 1 件、3 件、5 件から指定できる。出力は類似度順の PC 構成であり、各候補についてカテゴリ、登録用途、CPU、GPU、RAM、SSD、電源、概算価格、推薦理由、予算残額を表示する。

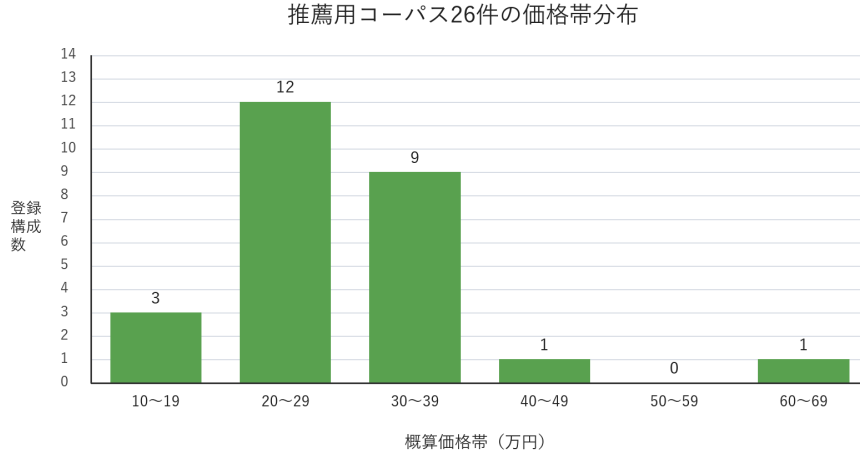


Figure 1: 推薦用コーパス 26 件の価格帯分布

CSV の各列と処理上の役割を表 1 に示す。文章埋め込みへ入力する列、予算判定に用いる列、結果表示だけに用いる列を分離することで、意味検索と構造化条件を混同しないようにした。

Table 1: PC 構成コーパスの列と利用方法

| 列             | データの内容   | システムでの利用方法      |
|---------------|----------|-----------------|
| use_case      | 用途を表す自然文 | SBERT による埋め込み計算 |
| price_yen     | 概算価格 (円) | 予算フィルタと残額計算     |
| cpu, gpu      | 主要部品     | 推薦結果への表示        |
| ram, ssd, psu | 容量・電源    | 推薦結果への表示        |
| category      | 評価用カテゴリ  | 正解率の計算と結果表示     |
| reason        | 構成の推薦理由  | 利用者への説明         |

表 1 のうち、検索順位を直接決めるのは `use_case` だけである。`price_yen` は候補を残すか除外するかの判定に使うが、予算内であれば価格の安さ自体は類似度へ加点しない。また、部品名や推薦理由を埋め込み対象へ含めないため、特定の GPU 名が入力文に偶然含まれたことだけで順位が決まる構成ではない。

入力文は自由記述とし、語数や定型句を要求しない。ただし、空文字では意味検索が成立しないため検索を実行しない。予算の下限を 14 万円としたのは、コーパス中の最安構成と対応させ、通常の画面操作で必ず一件以上の候補が残るようにするためである。候補数を 1, 3, 5 件に限定したのは、一件だけを素早く確認する利用と、複数案を比較する利用の両方を想定しつつ、情報過多を避けるためである。

### 3.2 処理フロー

システム全体の処理フローを図 2 に示す。

図 2 の各段階は、条件による抽出と意味による並べ替えに大別できる。最初の予算抽出で候補の行番号を保存し、その行に対応する埋め込みだけを類似度計算へ渡す。これにより、予算外の候補が高い類似度を得ても結果には入らない。その後、得点の上位から指定件数を選び、元の CSV 行と結合して画面表示用の辞書を作る。意味検索の結果と部品情報を同じ行番号で結び付けるため、類似度と異なる構成が表示されないようにしている。

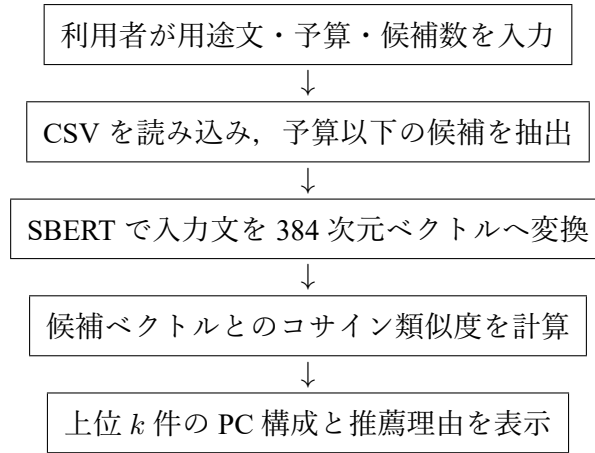


Figure 2: PC 構成推薦システムの処理フロー

### 3.3 モジュール構成

プロトタイプは Python と Streamlit で実装した。主な関数を表 2 に示す。

Table 2: 主な関数と役割

| 関数                                  | 役割                       |
|-------------------------------------|--------------------------|
| <code>load_data</code>              | CSV の読み込みと必須列の検証         |
| <code>load_model</code>             | Sentence-BERT モデルの読み込み   |
| <code>load_corpus_embeddings</code> | 全候補文の事前ベクトル化             |
| <code>recommend</code>              | 予算抽出，類似度計算，上位候補の生成       |
| <code>evaluate</code>               | 固定した 10 件による Top- $k$ 評価 |

モジュール間では，データ読み込み，モデル処理，推薦ロジック，画面表示を分離した。特に `recommend` は Streamlit の部品へ直接依存せず，入力値から推薦結果のリストを返す。そのため，同じ処理を検索画面と一括評価の両方から再利用できる。評価だけ別の推薦処理を実装すると，本番画面と異なる条件を測定する危険があるため，共通関数を用いる構成とした。

### 3.4 キャッシュ設計

モデルの読み込みと全文の埋め込み計算を検索のたびに行うと応答が遅くなる。そこで，モデルとコーパス埋め込みには `st.cache_resource` を適用した。CSV 読み込みには `st.cache_data` を適用した。初回検索ではモデル読み込みが必要だが，2 回目以降は入力文だけをベクトル化すればよい。

キャッシュは応答性を改善する一方，CSV を変更した際に古いデータや埋め込みが残る可能性がある。本プロトタイプでは開発中に Streamlit のキャッシュを消去することで対応する。実運用では，ファイルの更新日時やデータのハッシュ値をキャッシュキーへ含め，CSV と埋め込みの対応を保証する必要がある。

## 4 プロトタイプの実装

### 4.1 推薦処理

推薦処理では，最初に空文字，予算，候補数を検証する。次に，CSV の全候補から概算価格が予算以下の行番号だけを取得する。入力文を正規化済みベクトルに変換し，対象候補のベクトルとのコサイン類似度を一括計算する。最後に `topk` で上位候補を選び，類似度と予算残額を付加して返す。中核部分を以下に示す。

```

query_embedding = model.encode(
    query,
    convert_to_tensor=True,
    normalize_embeddings=True,
)
eligible_embeddings = corpus_embeddings[eligible_indices]
scores = util.cos_sim(query_embedding,
                       eligible_embeddings)[0]
top = scores.topk(k=min(top_k, len(eligible_indices)))

```

minを用いるのは、予算内候補が指定件数より少ない場合にも topkが範囲外エラーにならないようにするためである。また、検索後には概算価格を予算から引いて残額を計算する。残額は順位には使わないため、安い候補が自動的に上位になるわけではない。本システムにおける順位の第一基準は用途の類似度であり、予算は可否を決める条件である。この役割分担により、「予算を使い切る構成」と「安価で余裕を残す構成」を上位候補間で比較できる。

## 4.2 ユーザインタフェース

画面は Python から Web 画面を構築できる Streamlit[4] を使い、「構成を検索」と「性能評価」の二つのタブで構成した。検索タブでは、自然文入力、予算スライダー、候補数選択を一つの画面に配置した。結果は候補ごとの枠に分け、部品名と概算価格を表示する。類似度だけでなく、登録用途と推薦理由も表示することで、利用者が推薦結果を解釈しやすいようにした。

類似度だけを表示した場合、利用者はなぜその構成が選ばれたか判断しにくい。そこで登録用途を併記し、入力文とどの点が近いかを利用者自身が確認できるようにした。推薦理由は、部品の役割を短文で補足する情報であり、類似度から自動生成した文章ではない。あらかじめ CSV へ登録した説明を表示するため、同じ候補には常に同じ説明が示され、結果の再現性を保てる。

性能評価タブでは、ボタン一つで 10 件を評価し、Top-1 正解率、Top-3 正解率、各入力の期待カテゴリと予測カテゴリを表示する。これにより、推薦結果を目視するだけでなく、同じ条件で繰り返し比較できる。

## 4.3 入力検証と例外への対応

CSV 読み込み時には、用途、価格、部品、カテゴリ、推薦理由に対応する必須列が存在するかを検査する。必須列がない CSV をそのまま処理すると、検索後の画面表示で初めてエラーが発生し、原因が分かりにくい。そこで load\_data の段階で列構成を検査し、不正な場合は明示的に例外を発生させるようにした。

利用者の入力については、前後の空白を除去した後に空文字でないことを確認する。空文字の場合はモデルへ渡さず、画面上で用途の入力を求める警告を表示する。推薦関数単体でも、予算または候補数が 0 以下であれば空の結果を返す。画面部品による制限だけに依存せず、処理関数側でも値を検査することで、将来別の画面やテストコードから呼び出した場合の安全性を保っている。

予算内候補が存在しない場合も空リストを返し、画面側で候補がないことを表示する。また、上位件数は予算内候補数との小さい方を使用するため、候補が一件しかない状況で三件を要求しても topk の範囲外エラーにならない。このような例外処理は推薦精度そのものを高めるものではないが、実用的なプロトタイプとして利用者の操作で停止しないために必要である。

## 4.4 実行環境と再現手順

実装には Python, Streamlit, sentence-transformers, PyTorch, pandas を使用した。必要なライブラリは requirements.txt に記録し、起動コマンドは README へ記載した。初回実行時には

Hugging Face から学習済みモデルを取得するため通信が必要だが、モデル取得後の文章埋め込みと類似度計算はローカル環境で実行される。外部の生成 AI API へ用途文や PC 構成を送信しないため、API キーも不要である。

同じモデル、同じ CSV、同じ入力と予算を使用すれば、推論時の乱数を用いないため同じ類似度順位を再現できる。評価用の 10 件も TEST\_CASES として固定し、検索画面と同じ recommend 関数へ入力している。評価専用の別ロジックを用意しないことで、画面上の推薦と評価値の計算条件を一致させた。

#### 4.5 実行結果の例

用途文に「モンハンを WQHD の高画質かつ快適に遊びたい」、予算に 30 万円、表示候補数に 3 件を指定した実行画面を図 3 に示す。画面上では、入力欄と検索条件に続けて、類似度が最も高い候補の部品構成、概算価格、推薦理由、予算残額が一つのカードにまとめられている。

用途・予算別 PC 構成推薦

Sentence-BERT で入力文の意味を比較し、予算内の構成を推薦します。

構成を検索 性能評価

どの用途で使いますか？

モンハンを WQHD の高画質かつ快適に遊びたい

予算 (万円)

30

表示する候補数

3

おすすめ構成を検索

**候補1: WQHD ゲーム**

類似度 0.765 / 登録用途: WQHD でモンハンや AAA ゲームを覚悟で遊びたい

|               |                     |
|---------------|---------------------|
| CPU           | GPU                 |
| Ryzen 7 9700X | GeForce RTX 5070 Ti |
| ラム            | SSD                 |
| 32GB          | 2TB                 |
| 電源            | 任意価格                |
| 850W          | 260,000円            |

推薦理由: GPU性能を重視したWQHD向け。

予算残額: 40,000円

**候補2: 低予算eスポーツ**

類似度 0.580 / 登録用途: 低予算でVALORANTやlolをフルHDで遊びたい

Figure 3: PC 構成推薦の実行画面（入力条件と第 1 候補）

図 4 は同じ検索結果の続きであり、第 2 候補と第 3 候補の構成を示している。利用者は画面をスクロールすることで、上位候補の部品構成と価格を同じ形式で比較できる。

このときの上位 3 候補を表 3 に示す。1 位は「WQHD ゲーム」で、類似度は 0.765 であった。入力文に含まれる「モンハン」、「WQHD」、「高画質」という特徴が、登録用途の「WQHD でモンハンや AAA ゲームを高画質で遊びたい」と意味的に強く対応した結果である。

1 位の候補は用途との対応が明確であり、予算条件も満たした。一方、2 位には「低予算 e スポーツ」が選ばれた。これはゲーム用途という共通性を捉えたものの、WQHD 高画質という要求への適合度は低い。3 位の「WQHD 高 fps」は解像度の要求には合うが、高画質よりフレームレートを重視する登録文であるため、1 位より低くなった。この例から、SBERT は用



Figure 4: PC 構成推薦の実行画面（第 2 候補と第 3 候補）

Table 3: 実行画面における上位 3 候補

| 順位 | カテゴリ       | 類似度   | 主な構成                          | 価格        | 予算残額      |
|----|------------|-------|-------------------------------|-----------|-----------|
| 1  | WQHD ゲーム   | 0.765 | Ryzen 7 9700X / RTX 5070 Ti   | 260,000 円 | 40,000 円  |
| 2  | 低予算 e スポーツ | 0.580 | Ryzen 5 7500F / RTX 4060      | 140,000 円 | 160,000 円 |
| 3  | WQHD 高 fps | 0.534 | Ryzen 7 9800X3D / RTX 5070 Ti | 280,000 円 | 20,000 円  |



途の意味を反映して順位付けできる一方、類似度だけでは画質、解像度、フレームレートなどの個別条件を常に厳密に制約できるわけではないことが分かる。

#### 4.6 評価方法

CSV の用途文をそのまま入力すると評価が過度に容易になるため、意味を保った言い換え文を 10 件用意した。各入力には、正解と期待するカテゴリと、そのカテゴリの構成が含まれる予算を設定した。

評価指標は Top-1 正解率と Top-3 正解率である。テスト数を  $N$ 、正解カテゴリを  $y_i$ 、上位  $k$  件のカテゴリ集合を  $P_i^{(k)}$  とすると、Top- $k$  正解率を式 (2) で求める。

$$\text{Accuracy}@k = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[y_i \in P_i^{(k)}] \quad (2)$$

Top-1 は最初に提示される候補の正確さを表し、利用者が一件だけ確認する場面に対応する。Top-3 は比較候補まで含めて目的のカテゴリを発見できる割合であり、推薦システムとしての候補抽出能力を表す。各テストでは期待カテゴリの価格以上の予算を設定し、正解候補が予算フィルタで除外されない条件にした。したがって本評価は、価格設定の妥当性ではなく、予算内に存在する正解を意味検索で上位へ並べられるかを確認するものである。

比較可能性を持たせるため、全テストに同一モデル、同一コーパス、同一の候補数 3 件を使用した。ただし、10 件はカテゴリごとに一件ずつ無作為抽出した標本ではなく、代表的な利用場面を選んだ機能検証用データである。このため正解率に信頼区間を付けて一般化性能を主張するのではなく、想定した代表入力で機能が成立するかを確認する評価として位置付ける。

#### 4.7 評価結果

評価結果を表 4 に示す。

Table 4: 言い換え入力 10 件に対する評価結果

| 指標    | 正解数   | 正解率    |
|-------|-------|--------|
| Top-1 | 10/10 | 100.0% |
| Top-3 | 10/10 | 100.0% |

例えば「WQHD でゲームを快適に遊びたい」、予算 28 万円という入力に対し、1 位は「WQHD ゲーム」で、コサイン類似度は 0.883 であった。2 位は「競技 FPS」で 0.741、3 位は「低予算 e スポーツ」で 0.634 であり、用途の意味に対応した順位となった。

全 10 件で正解したことから、今回用意した用途の範囲では、登録文と完全一致しない入力でも意味的に近い構成を取得できた。一方、評価データは開発者が作成した 10 件のみであり、未知の利用者による評価ではない。したがって、100% という値を一般的な推薦性能として解釈することはできない。

各入力における 1 位候補の類似度を図 5 に示す。10 件の値は 0.580 から 0.931 の範囲であり、全件正解であっても入力によって確信度に差があることが分かる。VR ゲームは 0.931、WQHD ゲームは 0.910 と高かった一方、競技 FPS は 0.580、3D 制作は 0.687 にとどまった。したがって、カテゴリ一致率だけでなく、類似度の分布も併せて確認することで、表現の追加が必要な用途を発見できる。

また、Top-1 と Top-3 が同じ 100% であったため、今回の 10 件だけでは候補数を増やす効果を比較できなかった。これはシステムが完全であることを意味せず、評価例が既存カテゴリと明確に対応していた可能性が高い。実際の利用では、「安くて静かで AI も少し試したい」のような複合的で曖昧な文、誤字を含む文、コーパスに存在しないソフトウェア名が入力される。次の評価では、このような難易度別の入力群を用意し、Top-1 と Top-3 の差、類似度の分布、予算変更による順位変化を比較する必要がある。

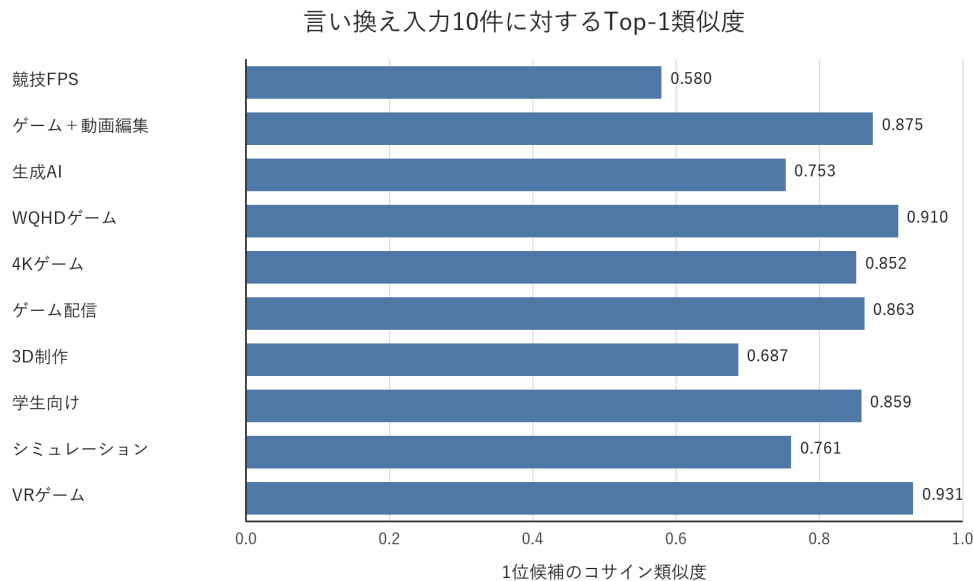


Figure 5: 言い換え入力 10 件に対する Top-1 類似度

## 5 最終的なシステムの考察

### 5.1 期待される成果

完成形では、利用者が部品名を知らなくても、用途と予算だけで比較の出発点となる PC 構成を得られる。キーワード一致ではなく文章埋め込みを使用するため、「動画編集」と「YouTube用の映像制作」のような言い換えにも対応できる。また、外部の生成 AI API に依存せず、ローカルで同じ入力に対して再現可能な類似度を計算できる。

販売員へ相談する前の自己整理や、複数の完成品 PC を比較する際の基準作りにも利用できる。利用者は推薦された部品を絶対的な正解として受け取るのではなく、登録用途と推薦理由を読み、自分の要求に近い点と不足する点を確認できる。管理者側は新しい用途や価格帯の構成を CSV へ追加することで、モデルを再学習せずに検索対象を更新できる。これは、少量の候補データから運用を開始し、利用記録を見ながら段階的に拡充する場合に有効である。

### 5.2 問題点

第一に、コーパスが 26 件と小さく、候補に存在しない用途や複合的な要求には弱い。類似度は文章同士の意味の近さを表すが、PC 部品間の互換性や、実際のフレームレートを保証する値ではない。

第二に、価格を固定値としているため、市場価格の変動や在庫を反映できない。また、「静音」「省電力」などを入力しても、候補文の情報だけで順位が決まり、消費電力や騒音の実測値を用いた制約判定は行っていない。

第三に、評価用入力と期待カテゴリを開発者が作成している。同じ人物がコーパスと評価データを作ることによって、表現やカテゴリに偏りが入る。実利用者が自由に入力する場合の誤記、長文、曖昧な要求については追加検証が必要である。

第四に、評価指標がカテゴリ一致だけであり、部品構成の品質や画面の使いやすさを直接測っていない。期待カテゴリが一致しても、予算残額が大きすぎる、特定ソフトウェアの推奨要件を満たさないなど、利用者にとって不十分な場合がある。逆に、期待カテゴリとは異なっても、部品構成としては要求を満たす候補もあり得る。したがってカテゴリ一致率は検索機能の一側面であり、推薦全体の品質を表す唯一の指標ではない。

### 5.3 改善案

今後は用途文と構成を増やし、複数人が作成した評価データを訓練・調整用と最終評価用に分離する。価格、消費電力、GPU メモリ、筐体サイズなどは文章類似度とは別の構造化条件として扱う。類似度が低い場合には無理に推薦せず、追加質問や「該当候補なし」を提示する閾値も必要である。

さらに、SBERT による検索結果を第一段階の候補抽出とし、第二段階で部品互換性と定量条件を規則ベースで検証する構成が考えられる。この二段階構成により、文章の柔軟な検索能力と、PC 構成に必要な厳密な制約判定を両立できる。

評価方法については、コーパス作成者とは別の協力者に入力文と妥当な候補を作成してもらい、少なくとも各カテゴリに複数例を用意する。さらに、検索成功率だけでなく、推薦理由の分かりやすさ、候補比較に要した時間、最終候補への納得度を5段階で回答してもらうユーザビリティ評価を組み合わせる。定量条件については、例えば解像度、目標フレームレート、必要 GPU メモリを個別項目として収集し、SBERT の得点と制約充足率を別々に表示する。

類似度の閾値は固定値を恣意的に決めるのではなく、正解例と不正解例のスコア分布を評価データから求めて設定することが望ましい。閾値未満の場合には追加質問を表示し、「ゲームなら解像度と重視する画質を入力してください」のように不足情報を補う。これにより、候補が少ないにもかかわらず最も近い一件を強制的に推薦する問題を抑えられる。

### 5.4 まとめ

本課題では、多言語 Sentence-BERT を用いて、利用者の用途文と PC 構成の登録用途を 384 次元の文章埋め込みへ変換し、コサイン類似度と予算条件から候補を推薦するプロトタイプを実装した。固定した言い換え入力 10 件では Top-1, Top-3 とともに 100% となり、小規模な対象範囲で意味検索が機能することを確認した。今後はコーパスと独立評価データの拡充、価格情報の更新、部品互換性の検証を追加する必要がある。

## References

- [1] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” Proceedings of EMNLP-IJCNLP 2019, pp. 3982–3992, 2019. <https://arxiv.org/abs/1908.10084>
- [2] Sentence Transformers, “paraphrase-multilingual-MiniLM-L12-v2 Model Card,” <https://huggingface.co/sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2>, 閲覧日：2026 年 7 月 25 日.
- [3] Sentence Transformers, “Semantic Textual Similarity,” [https://sbert.net/docs/sentence\\_transformer/usage/semantic\\_textual\\_similarity.html](https://sbert.net/docs/sentence_transformer/usage/semantic_textual_similarity.html), 閲覧日：2026 年 7 月 25 日.
- [4] Streamlit, “Streamlit documentation,” <https://docs.streamlit.io/>, 閲覧日：2026 年 7 月 25 日.